

MCP (Model Context Protocol)

MCP (Model Context Protocol) is becoming important because modern AI systems need a **standard way to connect LLMs with external tools, data sources, and applications.**

Think of MCP as a **USB-C port for AI models** 🔌 .

Instead of writing custom integrations for every tool, MCP provides **one standard protocol** that lets AI interact with many tools.

1 Why MCP is Needed

Before MCP, every AI system had **custom integrations.**

Example without MCP:

LLM → Custom code → Database

LLM → Custom code → Slack

LLM → Custom code → GitHub

LLM → Custom code → CRM

Problems ❌

- Every integration is **different**
- Hard to scale
- Hard to maintain
- Hard to reuse tools across models

With MCP

```
graph TD
  LLM[LLM] --> MCPClient[MCP Client]
  MCPClient --> MCPServer[MCP Server]
  MCPServer --> Tools[Tools / APIs / Databases]
```

Benefits ✅

- Standard communication protocol
- Reusable tools

- Easier integrations
- Works across different AI models
- Safer tool access

2 Simple Analogy (Real Life)

Think of **MCP** like an electrical adapter  .

Device	Role
Laptop	AI model
Adapter	MCP
Wall sockets	External tools

Without adapter → laptop cannot connect to different sockets.

MCP allows AI to **connect to any tool easily**.

3 Practical Life Example of MCP

Example: AI Software Engineer

Imagine an AI assistant helping a developer.

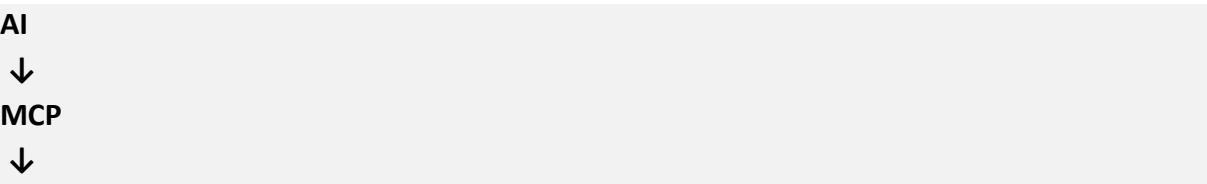
The AI needs to:

- 1 Read code from GitHub
- 2 Check errors in logs
- 3 Run terminal commands
- 4 Search documentation

Without MCP:

- AI → Custom GitHub API code
- AI → Custom terminal integration
- AI → Custom log system

With MCP:



GitHub Tool
Terminal Tool
Logs Tool
Docs Tool

The AI **does not need custom integration** for each tool.

4 Real Enterprise Example

Example: **Bank customer support AI**

User asks:

"Why was my loan application rejected?"

The AI may need to access:

- CRM database
- Loan policy documents
- Credit score API
- Customer history

Using MCP:

Customer Question



LLM



MCP



CRM Tool

Loan Policy Tool

Credit Score Tool

The AI can **securely call tools via MCP**.

5 Difference: MCP vs RAG vs AI Agents

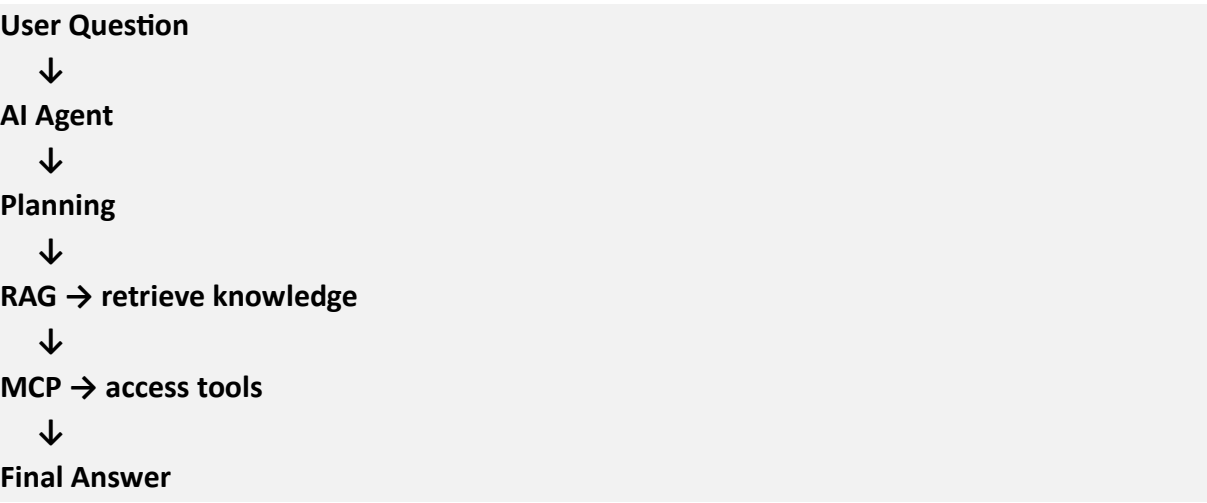
Feature	RAG	AI Agents	MCP
Purpose	Add knowledge	Execute tasks	Connect tools
Focus	Retrieval	Planning & acting	Standard communication

Feature	RAG	AI Agents	MCP
Data	Documents	Tools + memory	Tools/APIs
Example	PDF chatbot	Auto research agent	Tool integration layer

6 How They Work Together (Modern AI System)

Modern systems combine **all three**.

Example architecture:



Example:

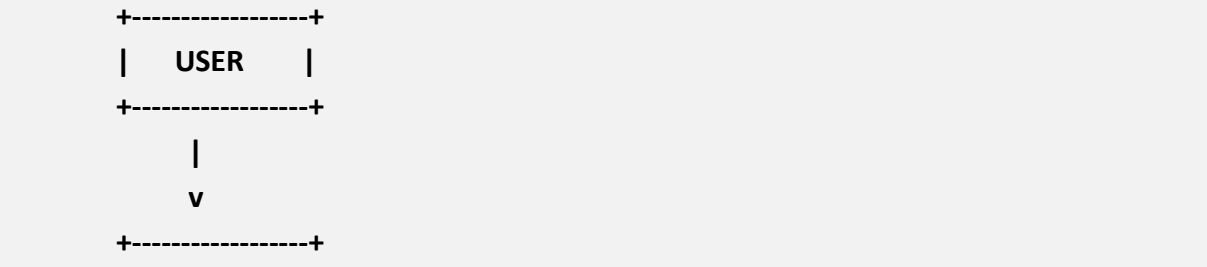
User:

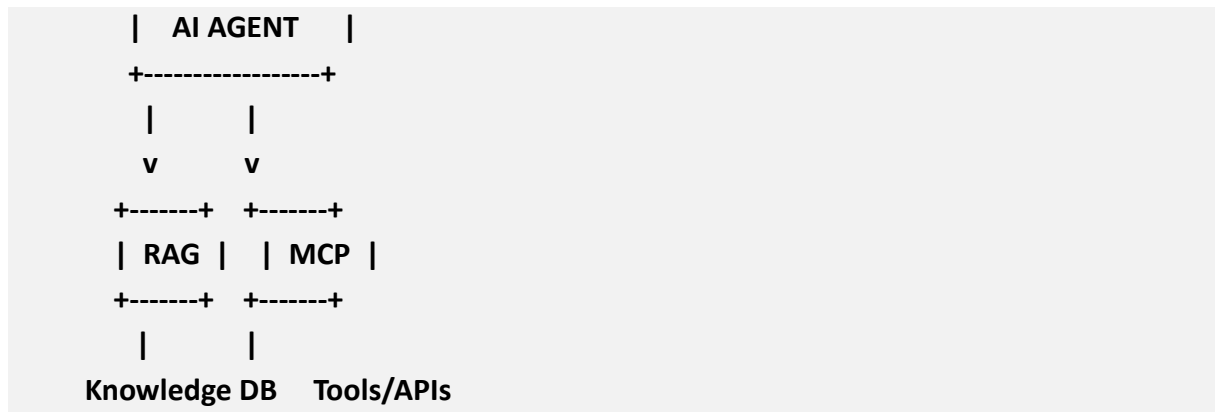
"What is the current status of order #1234?"

Agent does:

- 1 Retrieve order policy (RAG)
- 2 Call order API (MCP)
- 3 Decide next action (Agent)

7 Simple Visualization





8 Real Tools Using MCP

Examples of MCP servers/tools:

- GitHub MCP server
- Google Drive MCP server
- Slack MCP server
- Database MCP server
- File system MCP server

9 One-Line Summary

Technology Meaning

RAG	Gives AI knowledge
Agents	Give AI decision-making
MCP	Gives AI tool connectivity

✓ Best way to remember

RAG = Brain Knowledge

Agent = Brain Thinking

MCP = Hands to interact with world

1 How MCP Works with LangChain / AutoGen

Core Idea

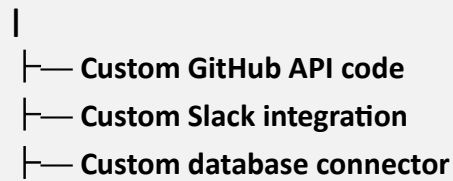
Frameworks like **LangChain** and **Microsoft AutoGen** allow LLMs to **use tools**.

MCP standardizes how those tools are connected.

Without MCP

Each tool requires **custom integration**.

LangChain Agent

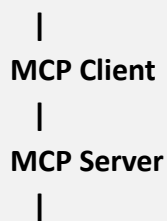


Problems

- Hard to maintain
 - Code duplication
 - Different authentication systems
 - Not reusable across models
-

With MCP

LangChain Agent



| GitHub | Slack | DB |

Now the **agent only talks to MCP**, not individual tools.

Example Flow

User asks:

"Summarize the latest commit from GitHub repo."

System flow:



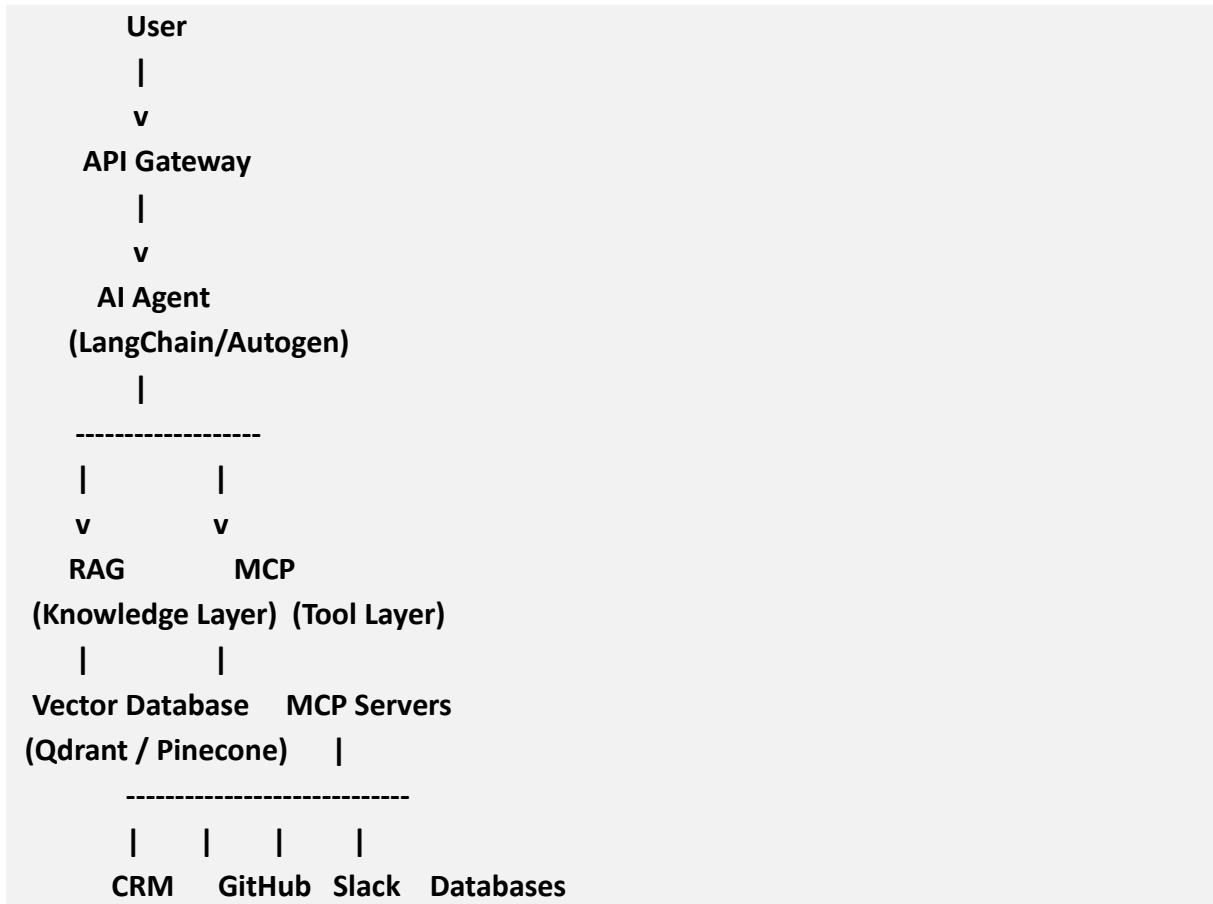
The same MCP server works with

- LangChain
- AutoGen
- Claude
- OpenAI
- local LLMs

2 Real MCP Architecture Used by Companies

Modern AI systems are becoming **tool-centric architectures**.

Typical enterprise setup:



Example technologies used:

Vector DB

→ Qdrant

→ Pinecone

Agent frameworks

→ LangChain

→ Microsoft AutoGen

MCP providers

→ Anthropic created MCP.

3 How to Build an MCP Server in Python

Basic idea:

An **MCP server exposes tools** that AI models can call.

Example tool: **weather API**

Step 1 Install MCP SDK

```
pip install mcp
```

Step 2 Create MCP Server

```
from mcp.server import Server

server = Server("weather-server")

@server.tool()
def get_weather(city: str) -> str:
    return f"The weather in {city} is 28°C"

if __name__ == "__main__":
    server.run()
```

Step 3 MCP Tool Metadata

MCP automatically exposes:

```
Tool name: get_weather
Input:
  city: string
Output:
  weather information
```

LLM can now **discover and use this tool dynamically**.

Step 4 Agent Using MCP

Example with **LangChain**

```
agent = Agent(  
    model="gpt-4",  
    tools_from_mcp="weather-server"  
)  
  
agent.run("What is weather in Delhi?")
```

Flow

Agent



MCP Client



Weather MCP Server



Tool execution



MCP vs API vs Plugin (Important Interview Question)

Feature	API	Plugin	MCP
Definition	Service endpoint	App extension	AI tool protocol
Standardization	No	Platform specific	Universal
Discovery	Manual	Platform managed	Automatic
AI Integration	Custom code	Limited	Native
Reusable across models	No	No	Yes

API Example

GET /weather?city=Delhi

LLM cannot understand this automatically.

Developer must write code.

Plugin Example

Used in older **ChatGPT plugin system.

Problems

- Platform specific
- Limited portability

MCP Example

Tool: get_weather

Input:

city

Output:

temperature

Any LLM can use it.

5 Simple Comparison

Think of it like this:

Technology	Role
API	raw function
Plugin	app extension
MCP	universal AI tool system

6 Future of MCP

Many companies are adopting MCP for **AI infrastructure**.

Possible ecosystem:

MCP Tool Marketplace

|

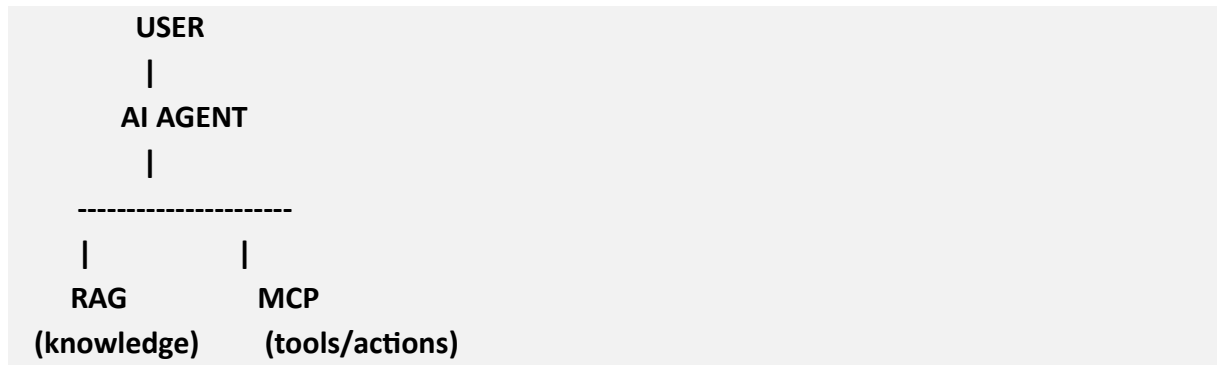
|

| GitHub MCP | Slack MCP | DB |

Agents simply connect and use tools.

7 Very Important Architecture Insight

Future AI stack will look like:



RAG = information

MCP = actions